

GordonAI

GordonAI is an AI package designed for sentiment analysis, emotion detection, and fact-checking classification. The models are pre-trained on three languages: Italian, English, and Spanish. This model has been trained specifically for fact-checking tasks. It classifies text into one of four categories: Disinformation, Hoax, FakeNews, or TrueNews. Based on the pre-trained mdeberta-v3-base model from Microsoft, it has been fine-tuned on a specialized fact-checking dataset to accurately identify whether a statement is true or false, and to detect misleading or fabricated information. You can use the GordonAI to classify texts helping to identify whether a statement is reliable or misleading.

The mdeberta-v3-base model

[DeBERTa](#) improves the BERT and RoBERTa models using disentangled attention and enhanced mask decoder. With those two improvements, DeBERTa outperform RoBERTa on a majority of NLU tasks with 80GB training data.

In [DeBERTa V3](#), we further improved the efficiency of DeBERTa using ELECTRA-Style pre-training with Gradient Disentangled Embedding Sharing. Compared to DeBERTa, our V3 version significantly improves the model performance on downstream tasks. You can find more technique details about the new model from our [paper](#).

Please check the [official repository](#) for more implementation details and updates.

mDeBERTa is multilingual version of DeBERTa which use the same structure as DeBERTa and was trained with CC100 multilingual data. The mDeBERTa V3 base model comes with 12 layers and a hidden size of 768. It has 86M backbone parameters with a vocabulary containing 250K tokens which introduces 190M parameters in the Embedding layer. This model was trained using the 2.5T CC100 data as XLM-R.

DeBERTa improves the BERT and RoBERTa models using disentangled attention and enhanced mask decoder. With those two improvements, DeBERTa outperform RoBERTa on a majority of NLU tasks with 80GB training data.

In DeBERTa V3, we further improved the efficiency of DeBERTa using ELECTRA-Style pre-training with Gradient Disentangled Embedding Sharing. Compared to DeBERTa, our V3 version significantly improves the model performance on downstream tasks. You can find more technique details about the new model from our [paper](#).

Please check the [official repository](#) for more implementation details and updates.

mDeBERTa is multilingual version of DeBERTa which use the same structure as DeBERTa and was trained with CC100 multilingual data. The mDeBERTa V3 base model comes with 12 layers and a hidden size of 768. It has 86M backbone parameters with a vocabulary containing 250K tokens which introduces 190M parameters in the Embedding layer. This model was trained using the 2.5T CC100 data as XLM-R.

The original DeBERTa: Decoding-enhanced BERT with Disentangled Attention

Recent progress in pre-trained neural language models has significantly improved the performance of many natural language processing (NLP) tasks. In this paper we propose a new model architecture DeBERTa (Decoding-enhanced BERT with disentangled attention) that improves the BERT and RoBERTa models using two novel techniques. The first is the disentangled attention mechanism, where each word is represented using two vectors that encode its content and position, respectively, and the attention weights among words are computed using disentangled matrices on their contents and relative positions, respectively. Second, an enhanced mask decoder is used to incorporate absolute positions in the decoding layer to predict the masked tokens in model pre-training. In addition, a new virtual adversarial training method is used for fine-tuning to improve models' generalization. We show that these techniques significantly improve the efficiency of model pre-training and the performance of both natural language understanding (NLU) and natural language generation (NLG) downstream tasks. Compared to RoBERTa-Large, a DeBERTa model trained on half of the training data performs consistently better on a wide range of NLP tasks, achieving improvements on MNLI by +0.9% (90.2% vs. 91.1%), on SQuAD v2.0 by +2.3% (88.4% vs. 90.7%) and RACE by +3.6% (83.2% vs. 86.8%). Notably, we scale up DeBERTa by training a larger version that consists of 48 Transform layers with 1.5 billion parameters. The significant performance boost makes the single DeBERTa model surpass the human performance on the SuperGLUE benchmark (Wang et al., 2019a) for the first time in terms of macro-average score (89.9 versus 89.8), and the ensemble DeBERTa model sits atop the SuperGLUE leaderboard as of January 6, 2021, out performing the human baseline by a decent margin (90.3 versus 89.8).

Enhanced mask decoder. Like BERT, DeBERTa is pre-trained using masked language modeling (MLM). MLM is a fill-in-the-blank task, where a model is taught to use the words surrounding a mask token to predict what the masked word should be. DeBERTa uses the content and position information of the context words for MLM. The disentangled attention mechanism already considers the contents and relative positions of the context words, but

not the absolute positions of these words, which in many cases are crucial for the prediction. Consider the sentence “a new store opened beside the new mall” with the italicized words “store” and “mall” masked for prediction. Although the local contexts of the two words are similar, they play different syntactic roles in the sentence. (Here, the subject of the sentence is “store” not “mall,” for example.) These syntactical nuances depend, to a large degree, upon the words’ absolute positions in the sentence, and so it is important to account for a word’s absolute position in the language modeling process. DeBERTa incorporates absolute word position embeddings right before the softmax layer where the model decodes the masked words based on the aggregated contextual embeddings of word contents and positions. In addition, we propose a new virtual adversarial training method for fine-tuning PLMs to downstream NLP tasks. The method is effective in improving models’ generalization. We show through a comprehensive empirical study that these techniques substantially improve the efficiency of pre-training and the performance of downstream tasks. In the NLU tasks, compared to RoBERTa-Large, a DeBERTa model trained on half the training data performs consistently better on a wide range of NLP tasks, achieving improvements on MNLI by +0.9% (90.2% vs. 91.1%), on SQuADv2.0 by +2.3% (88.4% vs. 90.7%), and RACE by +3.6% (83.2% vs. 86.8%). In the NLG tasks, DeBERTa reduces the perplexity from 21.6 to 19.5 on the Wikitext-103 dataset. We further scale up DeBERTa by pre-training a larger model that consists of 48 Transformer layers with 1.5 billion parameters. The single 1.5B-parameter DeBERTa model substantially outperforms T5 with 11 billion parameters on the SuperGLUE benchmark (Wang et al., 2019a) by 0.6% (89.3% vs. 89.9%), and surpasses the human baseline (89.9 vs. 89.8) for the first time. The ensemble DeBERTa model sits atop the SuperGLUE leaderboard as of January 6, 2021, outperforming the human baseline by a decent margin (90.3 versus 89.8).

A Transformer-based language model is composed of stacked Transformer blocks (Vaswani et al., 2017). Each block contains a multi-head self-attention layer followed by a fully connected positional feed-forward network. The standard self-attention mechanism lacks a natural way to encode word position information. Thus, existing approaches add a positional bias to each input word embedding so that each input word is represented by a vector whose value depends on its content and position. The positional bias can be implemented using absolute position embedding (Vaswani et al., 2017; Radford et al., 2019; Devlin et al., 2019) or relative position embedding (Huang et al., 2018; Yang et al., 2019). It has been shown that relative position representations are more effective for natural language understanding and generation tasks (Dai et al., 2019; Shaw et al., 2018). The proposed disentangled attention mechanism differs from all existing approaches in that we represent each input word using two separate vectors that encode a word’s content and position, respectively, and attention weights among words are computed using

disentangled matrices on their contents and relative positions, respectively. attention weights among words are computed using disentangled matrices on their contents and relative positions, respectively.

THE DEBERTA ARCHITECTURE

For a token at position i in a sequence, we represent it using two vectors, t_{Hiu} and t_{Pijju} , which represent its content and relative position with the token at position j , respectively. The calculation of the cross attention score between tokens i and j can be decomposed into four components as $A_{ij} = t_{HiPijju} t_{HjPjjiu} H_i H_j H_i P_{jji} P_{ijjHj} P_{ijjP_{jji}}$ (2) That is, the attention weight of a word pair can be computed as a sum of four attention scores using disentangled matrices on their contents and positions as content-to-content, content-to-position, position-to-content, and position-to-position. Existing approaches to relative position encoding use a separate embedding matrix to compute the relative position bias in computing attention weights (Shaw et al., 2018; Huang et al., 2018). This is equivalent to computing the attention weights using only the content-to-content and content-to position terms in equation 2. We argue that the position-to-content term is also important since the attention weight of a word pair depends not only on their contents but on their relative positions, which can only be fully modeled using both the content-to-position and position-to-content terms. Since we use relative position embedding, the position-to-position term does not provide much additional information and is removed from equation 2 in our implementation. Taking single-head attention as an example, the standard self-attention operation (Vaswani et al., 2017) can be formulated as: $Q H W_q K H W_k V H W_v A Q K$? $d \text{ Ho softmax} p A q V$ where $H \in \mathbb{R}^{P \times N \times d}$ represents the input hidden vectors, $H_o \in \mathbb{R}^{P \times N \times d}$ the output of self-attention, $W_q, W_k, W_v \in \mathbb{R}^{d \times d}$ the projection matrices, $A \in \mathbb{R}^{P \times N \times N}$ the attention matrix, N the length of the input sequence, and d the dimension of hidden states. Denote k as the maximum relative distance, $p_{ij} = j - i$ as the relative distance from token i to token j , which is defined as: $\# p_{ij} = 0$ for $i = j$, $2k + 1$ for $i < j$, k for $i > j$ others k (3) For a token at position i in a sequence, we represent it using two vectors, t_{Hiu} and t_{Pijju} , which represent its content and relative position with the token at position j , respectively. The calculation of the cross attention score between tokens i and j can be decomposed into four components as $A_{ij} = t_{HiPijju} t_{HjPjjiu} H_i H_j H_i P_{jji} P_{ijjHj} P_{ijjP_{jji}}$ (2) That is, the attention weight of a word pair can be computed as a sum of four attention scores using disentangled matrices on their contents and positions as content-to-content, content-to-position, position-to-content, and position-to-position. Existing approaches to relative position encoding use a separate embedding matrix to compute the relative position bias in computing attention weights (Shaw et al., 2018; Huang et al., 2018). This is equivalent to computing the attention weights using only the content-to-content and content-to position terms in equation 2. We argue that the position-to-content term is also important since the

attention weight of a word pair depends not only on their contents but on their relative positions, which can only be fully modeled using both the content-to-position and position-to-content terms. Since we use relative position embedding, the position-to-position term does not provide much additional information and is removed from equation 2 in our implementation. Taking single-head attention as an example, the standard self-attention operation (Vaswani et al., 2017) can be formulated as: $Q H W_Q K H W_K V H W_V A Q K$ where $H \in \mathbb{R}^{N \times d}$ represents the input hidden vectors, $H_o \in \mathbb{R}^{N \times d}$ the output of self-attention, $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ the projection matrices, $A \in \mathbb{R}^{N \times N}$ the attention matrix, N the length of the input sequence, and d the dimension of hidden states. Denote k as the maximum relative distance, $p_{ij} \in \mathbb{R}^{0 \leq p_{ij} \leq k}$ as the relative distance from token i to token j , which is defined as: $p_{ij} = \begin{cases} j - i & \text{if } |j - i| \leq k \\ 0 & \text{otherwise} \end{cases}$ (3) We can represent the disentangled self-attention with relative position bias as equation 4, where Q_c, K_c and V_c are the projected content vectors generated using projection matrices $W_{Qc}, W_{Kc}, W_{Vc} \in \mathbb{R}^{d \times d}$ respectively, $P \in \mathbb{R}^{k \times d}$ represents the relative position embedding vectors shared across all layers (i.e., staying fixed during forward propagation), and Q_r and K_r are projected relative position vectors generated using projection matrices $W_{Qr}, W_{Kr} \in \mathbb{R}^{d \times d}$, respectively. $Q_c H W_{Qc} K_c H W_{Kc} V_c H W_{Vc} Q_r P W_{Qr} K_r P W_{Kr} A_{ij} = Q_c[i] K_c[j] + \text{loooooomooooon}$ (a) content-to-content $H_o \text{softmax} P A + Q_c[i] K_r[p_{ij}] + \text{loooooomooooon}$ (b) content-to-position $K_c[j] Q_r[p_{ij}] + \text{loooooomooooon}$ (c) position-to-content (4) $3d \times Q_c A_{ij}$ is the element of attention matrix A , representing the attention score from token i to token j . $Q_c[i]$ is the i -th row of Q_c . $K_c[j]$ is the j -th row of K_c . $K_r[p_{ij}]$ is the p_{ij} -th row of K_r with regarding to relative distance p_{ij} . $Q_r[p_{ij}]$ is the p_{ij} -th row of Q_r with regarding to relative distance p_{ij} . Note that we use p_{ij} rather than p_{ji} here. This is because for a given position i , position-to-content computes the attention weight of the key content at j with respect to the query position at i , thus the relative distance is p_{ij} . The position-to-content term is calculated as $K_c[j] Q_r[p_{ij}]$. The content-to-position term is calculated in a similar way. Finally, we apply a scaling factor of $\frac{1}{\sqrt{3d}}$ on A . The factor is important for stabilizing model training (Vaswani et al., 2017), especially for large-scale PLMs.

For an input sequence of length N , it requires a space complexity of $O(N^2 d)$ (Shaw et al., 2018; Huang et al., 2018; Dai et al., 2019) to store the relative position embedding for each token. However, taking content-to-position as an example, we note that since $p_{ij} \in \mathbb{R}^{0 \leq p_{ij} \leq k}$ and the embeddings $P \in \mathbb{R}^{k \times d}$ Published as a conference paper at ICLR 2021 of all possible relative positions are always a subset of $K_r \in \mathbb{R}^{k \times d}$, then we can reuse K_r in the attention calculation for all the queries. In our experiments, we set the maximum relative distance k to 512 for pre-training. The disentangled attention weights can be computed efficiently using Algorithm 1. Let R be the relative position matrix according to equation 3, i.e., $R_{ijs} = p_{ij}$. Instead of allocating a different relative position embedding matrix for each query, we

multiply each query vector Q_{c_i} by $K_r P_{rd} 2k$, as in line 3 5. Then, we extract the attention weight using the relative position matrix as the index, as in line 6 10. To compute the position-to-content attention score, we calculate $A_{p \tilde{N} c_r: js}$, i.e., the column vector of the attention matrix $A_{p \tilde{N} c}$, by multiplying each key vector K_{c_rj} by Q_r , as in line 11 13. Finally, we extract the corresponding attention score via the relative position matrix as the index, as in line 14 18. In this way, we do not need to allocate memory to store a relative position embedding for each query and thus reduce the space complexity to $O(pkdq)$ (for storing K_r and Q_r).

3.2 ENHANCED MASK DECODER ACCOUNTS FOR ABSOLUTE WORD POSITIONS

DeBERTa is pretrained using MLM, where a model is trained to use the words surrounding a mask token to predict what the masked word should be. DeBERTa uses the content and position information of the context words for MLM. The disentangled attention mechanism already considers the contents and relative positions of the context words, but not the absolute positions of these words, which in many cases are crucial for the prediction. Given a sentence “a new store opened beside the new mall” with the words “store” and “mall” masked for prediction. Using only the local context (e.g., relative positions and surrounding words) is insufficient for the model to distinguish store and mall in this sentence, since both follow the word new with the same relative positions. To address this limitation, the model needs to take into account absolute positions, as complement information to the relative positions. For example, the subject of the sentence is “store” not “mall”. These syntactical nuances depend, to a large degree, upon the words’ absolute positions in the sentence. There are two methods of incorporating absolute positions. The BERT model incorporates absolute positions in the input layer. In DeBERTa, we incorporate them right after all the Transformer layers but before the softmax layer for masked token prediction, as shown in Figure 2. In this way, DeBERTa captures the relative positions in all the Transformer layers and only uses absolute positions as complementary information when decoding the masked words. Thus, we call DeBERTa’s decoding component an Enhanced Mask Decoder (EMD). In the empirical study, we compare these two methods of incorporating absolute positions and observe that EMD works much better. We conjecture that the early incorporation of absolute positions used by BERT might undesirably hamper the model from learning sufficient information of relative positions. In addition, EMD also enables us to introduce other useful information, in addition to positions, for pre-training. We leave it to future work.

4 SCALE INVARIANT FINE-TUNING

This section presents a new virtual adversarial training algorithm, Scale-invariant-Fine-Tuning (SiFT), a variant to the algorithm described in Miyato et al. (2018); Jiang et al. (2020), for fine-tuning. Virtual adversarial training is a regularization method for improving models’ generalization. It does so by improving a model’s robustness to adversarial examples, which are created by making small perturbations to the input. The model is regularized so that when given a task-specific example, the model produces the same output distribution as it produces on

an adversarial perturbation of that example. For NLP tasks, the perturbation is applied to the word embedding instead of the original word sequence. However, the value ranges (norms) of the embedding vectors vary among different words and models. The variance gets larger for bigger models with billions of parameters, leading to some instability of adversarial training.

5 Published as a conference paper at ICLR 2021

Inspired by layer normalization (Ba et al., 2016), we propose the SiFT algorithm that improves the training stability by applying the perturbations to the normalized word embeddings. Specifically, when fine-tuning DeBERTa to a downstream NLP task in our experiments, SiFT first normalizes the word embedding vectors into stochastic vectors, and then applies the perturbation to the normalized embedding vectors. We find that the normalization substantially improves the performance of the fine-tuned models. The improvement is more prominent for larger DeBERTa models. Note that we only apply SiFT to DeBERTa15B on SuperGLUE tasks in our experiments and we will provide a more comprehensive study of SiFT in our future work.

For an input sequence of length N , it requires a space complexity of $O(N^2d)$ (Shaw et al., 2018; Huang et al., 2018; Dai et al., 2019) to store the relative position embedding for each token. However, taking content-to-position as an example, we note that since $p_{ij} = p_{r_0+2k}$ and the embeddings 4 Published as a conference paper at ICLR 2021 of all possible relative positions are always a subset of $K \times P \times R \times 2k$, then we can reuse K in the attention calculation for all the queries. In our experiments, we set the maximum relative distance k to 512 for pre-training. The disentangled attention weights can be computed efficiently using Algorithm 1.

Let $\mathbf{r}$ be the relative position matrix according to equation 3, i.e., $r_{ijs} = p_{ij}$. Instead of allocating a different relative position embedding matrix for each query, we multiply each query vector $\mathbf{Q}_{cri} : s$ by $K \times P \times R \times 2k$, as in line 3 5. Then, we extract the attention weight using the relative position matrix as the index, as in line 6 10. To compute the position-to-content attention score, we calculate $\mathbf{A}_{p\tilde{N}c} : js$, i.e., the column vector of the attention matrix $\mathbf{A}_{p\tilde{N}c}$, by multiplying each key vector $\mathbf{K}_{crj} : s$ by $\mathbf{Q}_r$, as in line 11 13. Finally, we extract the corresponding attention score via the relative position matrix as the index, as in line 14 18. In this way, we do not need to allocate memory to store a relative position embedding for each query and thus reduce the space complexity to $O(pkd)$ (for storing K and Q).

3.2 ENHANCED MASK DECODER ACCOUNTS FOR ABSOLUTE WORD POSITIONS

DeBERTa is pretrained using MLM, where a model is trained to use the words surrounding a mask token to predict what the masked word should be. DeBERTa uses the content and position information of the context words for MLM. The disentangled attention mechanism already considers the contents and relative positions of the context words, but not the absolute positions of these words, which in many cases are crucial for the prediction. Given a sentence “a new store opened beside the new mall” with the words “store” and “mall” masked for prediction. Using only the local context (e.g., relative positions and surrounding words) is insufficient for the model to

distinguish store and mall in this sentence, since both follow the word new with the same relative positions. To address this limitation, the model needs to take into account absolute positions, as complement information to the relative positions. For example, the subject of the sentence is “store” not “mall”. These syntactical nuances depend, to a large degree, upon the words’ absolute positions in the sentence. There are two methods of incorporating absolute positions. The BERT model incorporates absolute positions in the input layer. In DeBERTa, we incorporate them right after all the Transformer layers but before the softmax layer for masked token prediction, as shown in Figure 2. In this way, DeBERTa captures the relative positions in all the Transformer layers and only uses absolute positions as complementary information when decoding the masked words. Thus, we call DeBERTa’s decoding component an Enhanced Mask Decoder (EMD). In the empirical study, we compare these two methods of incorporating absolute positions and observe that EMD works much better. We conjecture that the early incorporation of absolute positions used by BERT might undesirably hamper the model from learning sufficient information of relative positions. In addition, EMD also enables us to introduce other useful information, in addition to positions, for pre-training. We leave it to future work.

4 SCALE INVARIANT FINE-TUNING

This section presents a new virtual adversarial training algorithm, Scale-invariant-Fine-Tuning (SiFT), a variant to the algorithm described in Miyato et al. (2018); Jiang et al. (2020), for fine-tuning. Virtual adversarial training is a regularization method for improving models’ generalization. It does so by improving a model’s robustness to adversarial examples, which are created by making small perturbations to the input. The model is regularized so that when given a task-specific example, the model produces the same output distribution as it produces on an adversarial perturbation of that example. For NLP tasks, the perturbation is applied to the word embedding instead of the original word sequence. However, the value ranges (norms) of the embedding vectors vary among different words and models. The variance gets larger for bigger models with billions of parameters, leading to some instability of adversarial training.

5 Published as a conference paper at ICLR 2021

Inspired by layer normalization (Ba et al., 2016), we propose the SiFT algorithm that improves the training stability by applying the perturbations to the normalized word embeddings. Specifically, when fine-tuning DeBERTa to a downstream NLP task in our experiments, SiFT first normalizes the word embedding vectors into stochastic vectors, and then applies the perturbation to the normalized embedding vectors. We find that the normalization substantially improves the performance of the fine-tuned models. The improvement is more prominent for larger DeBERTa models. Note that we only apply SiFT to DeBERTa15B on SuperGLUE tasks in our experiments and we will provide a more comprehensive study of SiFT in our future work